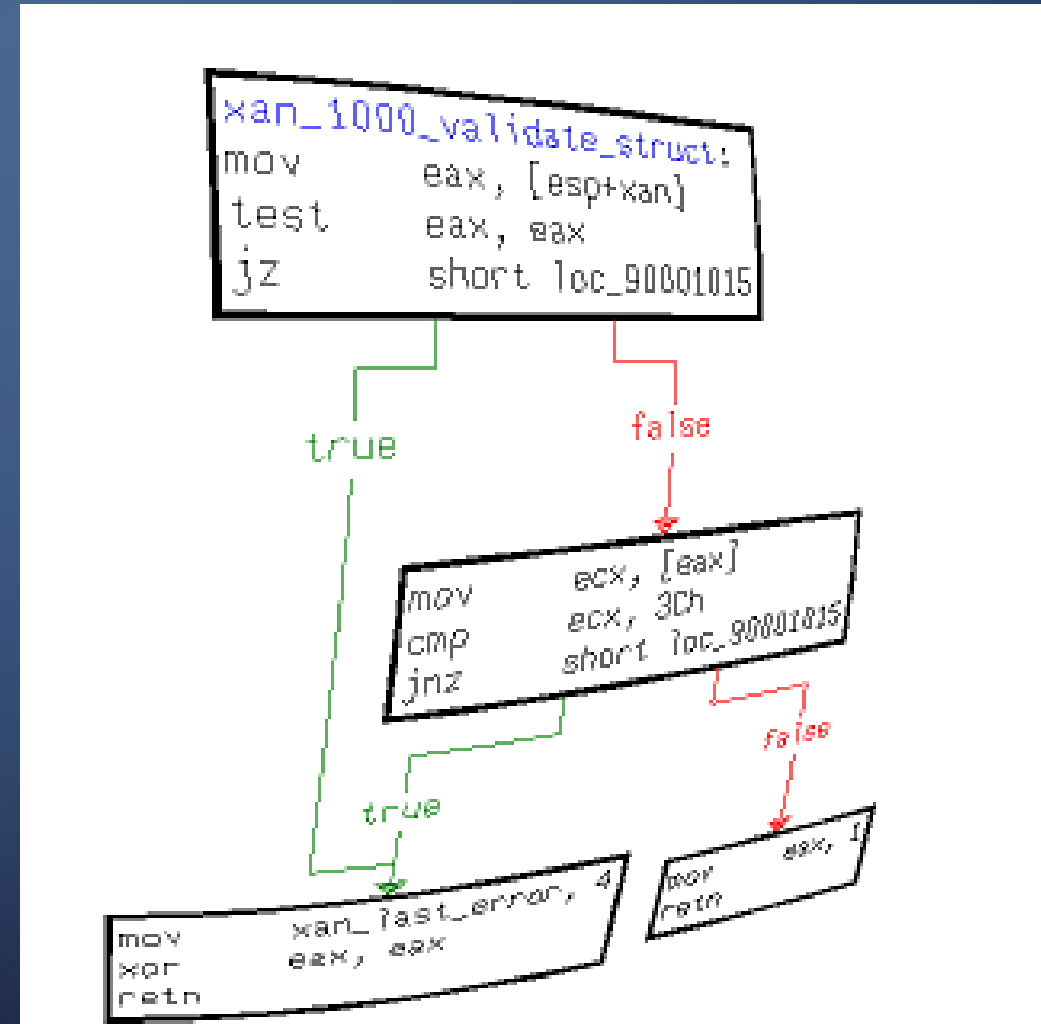


PROGRAM DEVELOPMENT, COMPILATION, AND REVERSE ENGINEERING USING PYTHON, C, AND IDA PRO

Hashan Kodippilige
Software Security Analysis - Project 1

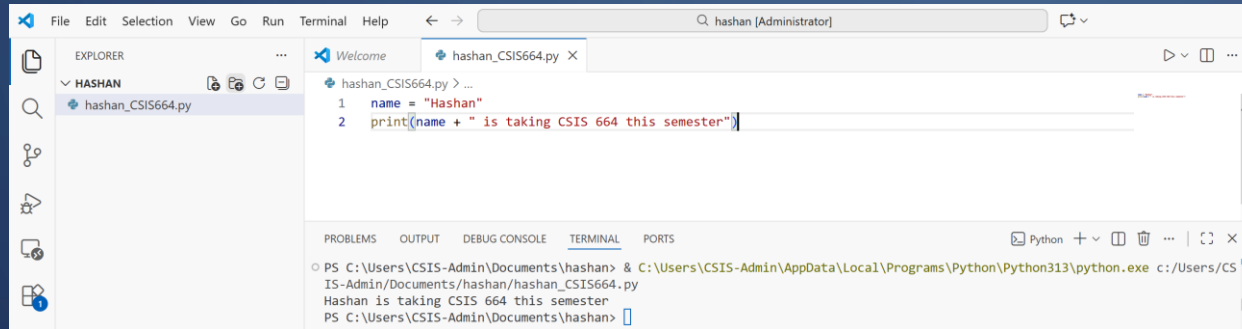


INTRODUCTION

- Software security analysts must understand how high-level source code is translated into executable files and how these interact with system-level libraries.
- This project focuses on developing, compiling, and analyzing programs using Python and C programming languages.
- A python program was converted into a Portable Executable (PE) file using PyInstaller, , while the C program was compiled and executed using the GCC compiler.
- IDA Pro was used to analyze executables, demonstrating how high-level programs are translated into machine-level operations and supported by system libraries.

PYTHON PROGRAM AND OUTPUT

- Python program developed using Visual Studio Code.
- Displays output as *"Hashan is taking CSIS 664 this semester"*



The screenshot displays the Visual Studio Code interface. The Explorer panel on the left shows a project named 'HASHAN' with a file 'hashan_CSIS664.py'. The main editor window shows the code for 'hashan_CSIS664.py':

```
1 name = "Hashan"
2 print(name + " is taking CSIS 664 this semester")
```

The bottom panel shows the TERMINAL output, which reads:

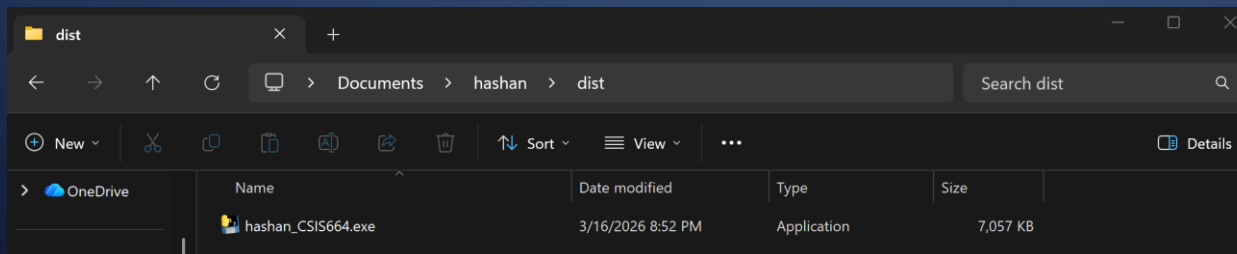
```
PS C:\Users\CSIS-Admin\Documents\hashan> & C:\Users\CSIS-Admin\AppData\Local\Programs\Python\Python313\python.exe c:/Users/CSIS-Admin/Documents/hashan/hashan_CSIS664.py
Hashan is taking CSIS 664 this semester
PS C:\Users\CSIS-Admin\Documents\hashan>
```

PYTHON PROGRAM .EXE FILE

- Python program converted into .exe file using PyInstaller.

```
PS C:\Users\CSIS-Admin\Documents\hashan> pyinstaller --onefile hashan_CSIS664.py
510 INFO: PyInstaller: 6.19.0, contrib hooks: 2026.1
511 INFO: Python: 3.13.1
632 INFO: Platform: Windows-11-10.0.26100-SP0
632 INFO: Python environment: C:\Users\CSIS-Admin\AppData\Local\Programs\Python\Python313
634 INFO: wrote C:\Users\CSIS-Admin\Documents\hashan\hashan_CSIS664.spec
659 INFO: Module search paths (PYTHONPATH):
['C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\Scripts\\pyinstaller.exe',
'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\python313.zip',
'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\DLLs',
'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\Lib',
'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313',
'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\site-packages',
'C:\\Users\\CSIS-Admin\\Documents\\hashan']
1268 INFO: checking Analysis
```

```
18711 INFO: Appending PKG archive to EXE
19063 INFO: Fixing EXE headers
19558 INFO: Building EXE from EXE-00.toc completed successfully.
19560 INFO: Build complete! The results are available in: C:\Users\CSIS-Admin\Documents\hashan\dist
```



C PROGRAM DEVELOPMENT

- C program developed using Ubuntu (WSL).
- Accepts user inputs
- Performs arithmetic calculations (sum)
- Uses if-else conditions for decision making

```
hashan@lab8K4EB1:~$ nano hashan_proj2.c
```

```
GNU nano 7.2 hashan_proj2.c
#include <stdio.h>

int main() {
    //Add integers
    int input1, input2, sum;

    // Prompt for user inputs
    printf("Enter input 1: ");
    scanf("%d", &input1);

    printf("Enter input 2: ");
    scanf("%d", &input2);

    // Get sum of input numbers
    sum = input1 + input2;

    // Display sum
    printf("Sum = %d\n", sum);

    // Conditions for sum
    if (sum < 10) {
        printf("The sum is less than 10\n");
    } else {
        printf("The sum is greater than (or equal) to 10\n");
    }

    return 0;
}
```

File Name to Write: hashan_proj2.c

^G Help	^M-D DOS Format	^M-A Append	^M-B Backup File
^C Cancel	^M-W Mac Format	^M-P Prepend	^M-T Browse

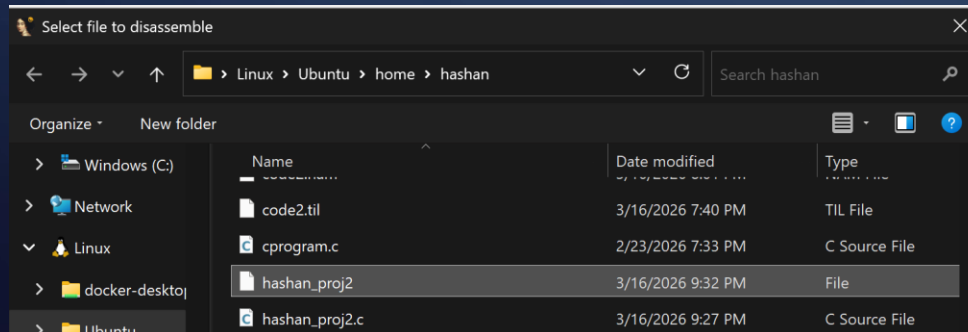
C PROGRAM OUTPUT AND EXECUTABLE FILE

- Program compiled using GCC compiler and captured the results with different inputs.

```
hashan@lab8K4EB1:~$ gcc hashan_proj2.c -o hashan_proj2
hashan@lab8K4EB1:~$ ./hashan_proj2
Enter input 1: 6
Enter input 2: 8
Sum = 14
The sum is greater than (or equal) to 10
```

```
hashan@lab8K4EB1:~$ ./hashan_proj2
Enter input 1: 4
Enter input 2: 5
Sum = 9
The sum is less than 10
```

```
hashan@lab8K4EB1:~$ ./hashan_proj2
Enter input 1: 6
Enter input 2: 4
Sum = 10
The sum is greater than (or equal) to 10
```

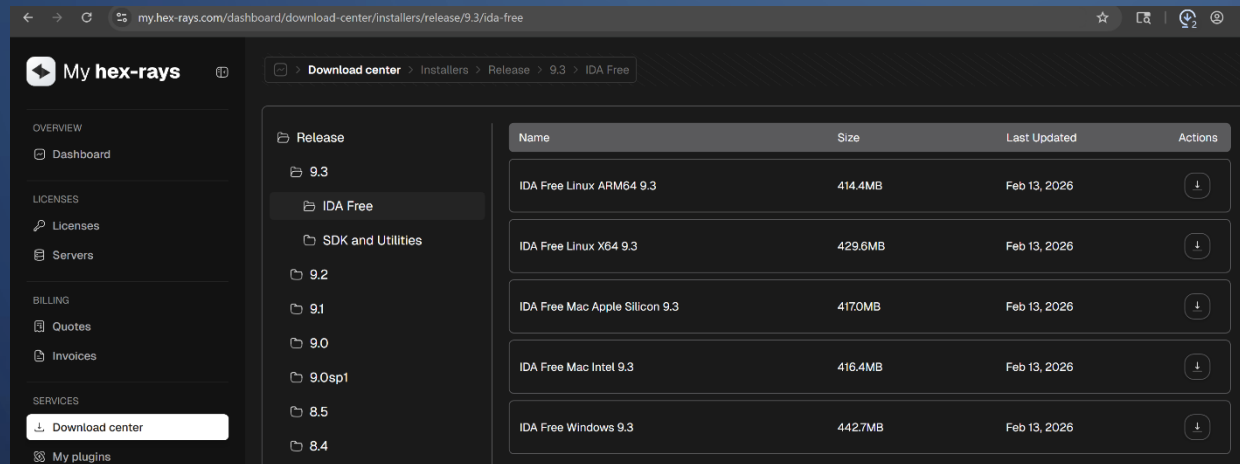


HOW TO AVOIDS COMMON PROGRAMMING WEAKNESS

- Structured the program clearly by separating input, processing, and output.
- To have accurate calculation, use appropriate data types, such as integers for numerical inputs.
- To reduce the input errors, user inputs are handled using `scanf()` function with correct format specifiers.
- Use *if-else* conditional structure to control the flow of execution.
- Provide the meaningful prompts to minimize the input errors, and easy to understand.

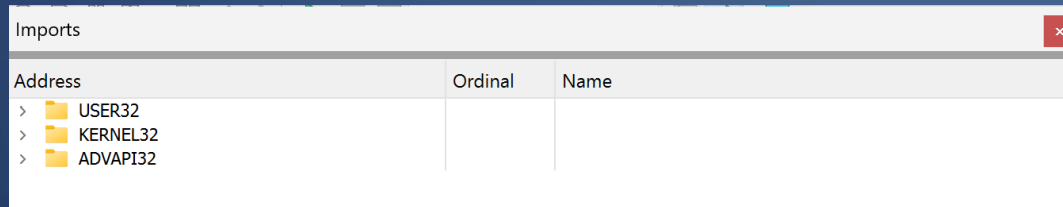
IDA PRO INSTALLATION

- Downloaded a free version of IDA Pro using URL <https://hex-rays.com/ida-free/#download> for Windows and installed it.

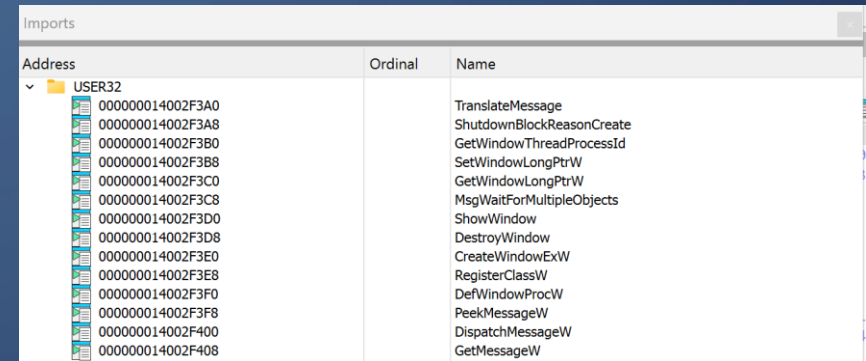


IDA ANALYSIS (PYTHON EXE – LIBRARIES)

- Imports window listed the external libraries used by the program, including USER32, KERNEL32, and ADVAPI32.



Address	Ordinal	Name
> USER32		
> KERNEL32		
> ADVAPI32		

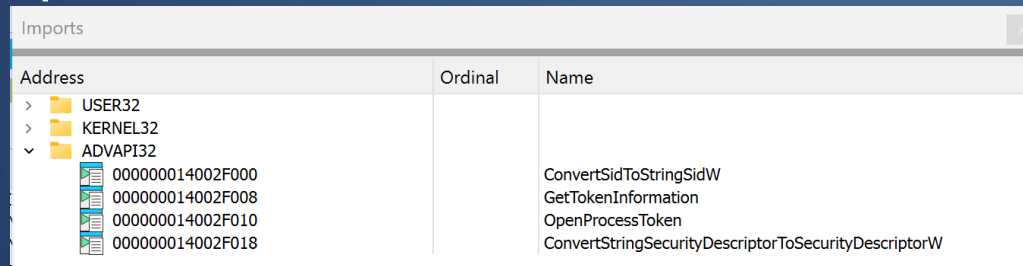


Address	Ordinal	Name
USER32		
000000014002F3A0		TranslateMessage
000000014002F3A8		ShutdownBlockReasonCreate
000000014002F3B0		GetWindowThreadProcessId
000000014002F3B8		SetWindowLongPtrW
000000014002F3C0		GetWindowLongPtrW
000000014002F3C8		MsgWaitForMultipleObjects
000000014002F3D0		ShowWindow
000000014002F3D8		DestroyWindow
000000014002F3E0		CreateWindowExW
000000014002F3E8		RegisterClassW
000000014002F3F0		DefWindowProcW
000000014002F3F8		PeekMessageW
000000014002F400		DispatchMessageW
000000014002F408		GetMessageW

- Main responsible of USER32 library includes manage user interface components and handle communication between the program and the Windows graphical system.
- USER32 provides window creation, message handling, and user interaction functions.
- Functions like ShowWindow, GetMessage, and DispatchMessage allow the program to interact with the Windows environment and respond to user actions or system events.

IDA ANALYSIS (PYTHON EXE – LIBRARIES)

- ADVAPI32 library displays the functions related to security and system-level operations.



The screenshot shows the 'Imports' window in IDA Pro. The 'Address' column lists four memory addresses under the 'ADVAPI32' library. The 'Ordinal' column is empty for all entries. The 'Name' column lists the corresponding function names: 'ConvertSidToStringSidW', 'GetTokenInformation', 'OpenProcessToken', and 'ConvertStringSecurityDescriptorToSecurityDescriptorW'.

Address	Ordinal	Name
> USER32		
> KERNEL32		
▼ ADVAPI32		
000000014002F000		ConvertSidToStringSidW
000000014002F008		GetTokenInformation
000000014002F010		OpenProcessToken
000000014002F018		ConvertStringSecurityDescriptorToSecurityDescriptorW

- The KERNEL32 library is a brain of the program's interaction with Windows operating system and it provides essential low-level system functionality required for program execution.
- This includes memory management, process and thread creation, file input/output operations, and system resource handling related functions.
- Functions like GetModuleHandle, CreateFile, ReadFile, and ExitProcess are commonly provided by KERNEL32 library, and used to manage program execution and system-level operations.

```

;
; +-----+
; | This file was generated by The Interactive Disassembler (IDA) |
; | Copyright (c) 2026 Hex-Rays, <support@hex-rays.com>          |
; | Freeware version                                           |
; +-----+
;
; Input SHA256 : 537590B6779B90E9F39A7E979175298F5D68ED79AC1E45CB8DA6AFD610
; Input MD5    : 9E020B423D34D9F779438DACE121B741
; Input CRC32  : F85EE923
; Compiler     : Visual C++ (guessed)
;
; File Name   : C:\Users\CSIS-Admin\Documents\hashan\dist\hashan_CSIS664.exe
; Format      : Portable executable for AMD64 (PE)
; Imagebase   : 140000000
; Timestamp   : 690803E1 (Tue Mar 17 01:52:33 2026 UTC)
; Section 1. (virtual address 00001000)
; Virtual size      : 0002D8A0 ( 186528.)
; Section size in file : 0002DA00 ( 186880.)
; Offset to raw data for section: 00000400
; Flags 60000020: Text Executable Readable
; Alignment        : default
; OS type          : MS Windows
; Application type: Executable
;
; .686p
; .mmx
; .model flat
;
; Segment type: Pure code
; Segment permissions: Read/Execute
; _text segment para public 'CODE' use64
; assume cs:_text
; org 140001000h
; assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing
;
; int __fastcall main(int argc, const char **argv, const char **envp)
; main proc near
; mov     rax, cs:off_1400309C8
; mov     [rax], ecx
; mov     rax, cs:off_1400309C8
; mov     [rax+8], rdx
; mov     rcx, cs:off_1400309C8
; jmp     sub_1400040E0
; main endp

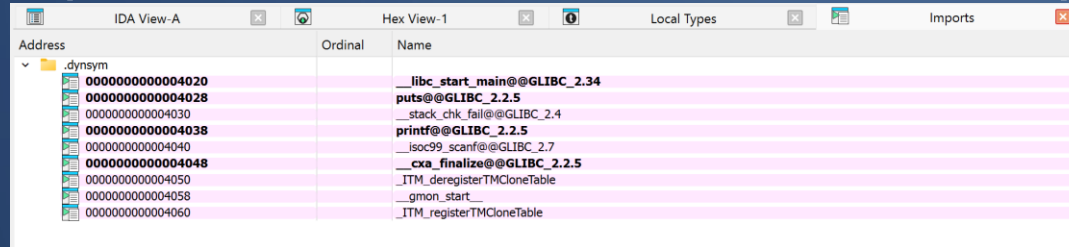
```

CONTROL FLOW GRAPH (PYTHON EXECUTABLE)

- A subroutine is a set of instructions used to perform a specific task and can be reused within a program.
- The main function is identified as the entry point of the executable.
- The function contains several low-level instructions, including mov, and jmp.
- The code initializes registers and prepares arguments for further processing.
- Values from registers such as ecx and rdx are stored in memory locations referenced by rax.

IDA ANALYSIS (C PROGRAM LIBRARIES)

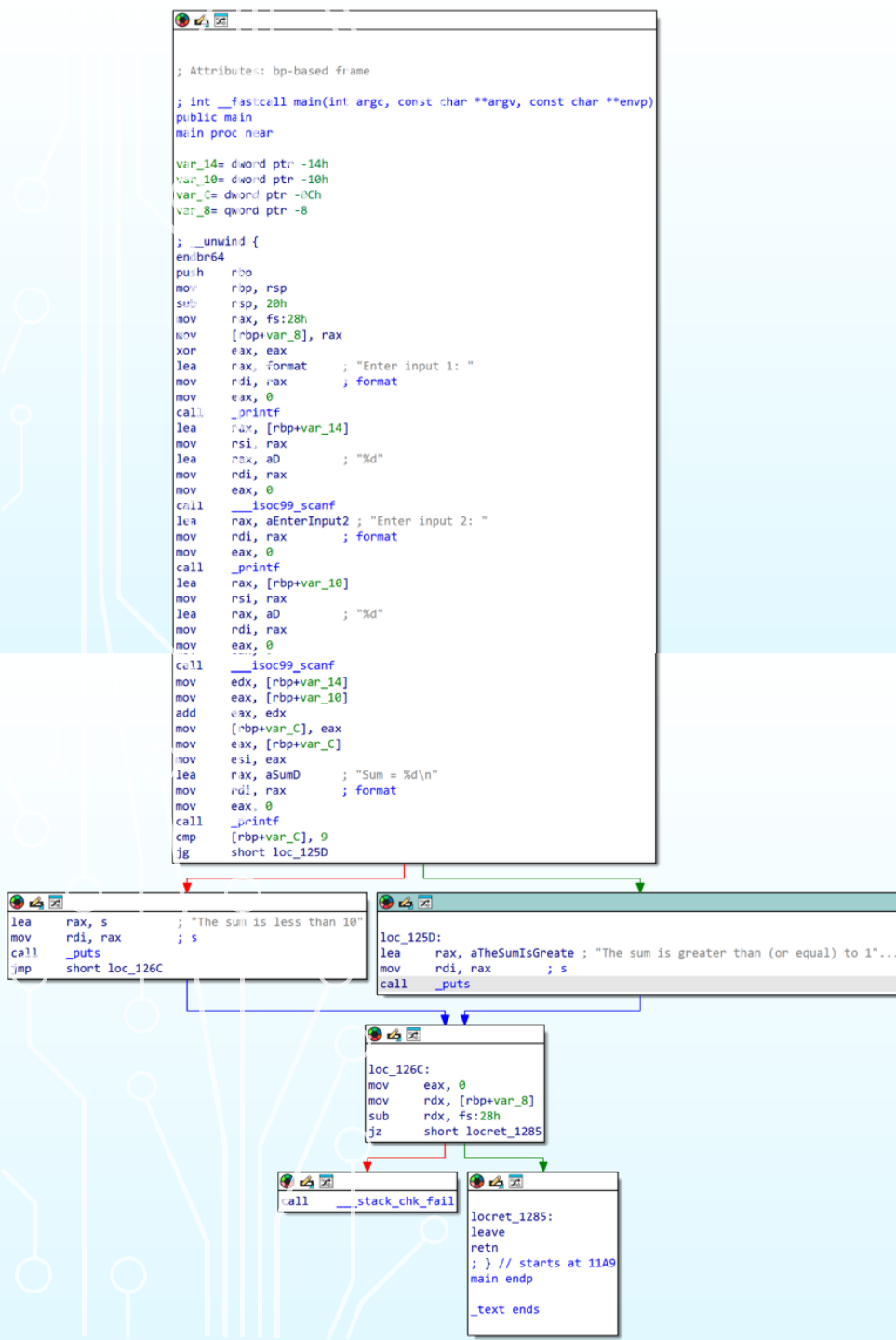
- Imports window listed the standard library functions used by the C program.



- This executable uses the GNU C Library (glibc). This library is the standard C library in Linux systems.
- Glibc library provides essential functionality required for input/output operations, program initialization, and interaction with the operating system.
- The *printf* and *scanf* function are part of the glibc library and essential for input and output operations in the program.
- The *__libc_start_main* function is serving as the entry point for C programs in a Linux environment.

CONTROL FLOW GRAPH (C PROGRAM)

- The main function is identified as the entry point of execution.
- Function setup includes push rbp and mov rbp, rsp which Prepares memory for local variables.
- To retrieve information values and compute their sum, the *mov* and *add* instructions were used.
- The result is compared with 10 using *cmp* instruction
- Program follows conditional branching based on comparison result.
- Each branch calls puts function to display output.
- Includes stack canary check for security



CONCLUSION

- To observe the lower-level function, we can compile and execute Python and C programming languages in a different environment.
- Python scripts can be transformed into standalone executables using the Pyinstaller while C programs use GCC compilation process.
- The IDA Pro can reveal the internal structure of executable files, including imported libraries and assembly-level instructions.
- Regardless of the size or complexity, program interact with system libraries such as *glibc* in Linux and API libraries like *USER32*, *KERNEL32*, and *ADVAPI32* in Windows.
- Disassembled code provides how functions, memory operations, and control flow are implemented at the machine level.

REFERENCES

- Hex-Rays. (2024). *IDA Freeware*.
<https://hex-rays.com/ida-free/#download>
- OTW. (2023, November 26). *Reverse engineering malware, part 03: IDA Pro introduction*. Hackers Arise.
<https://hackers-arise.com/reverse-engineering-malware-part-3-ida-pro-introduction/>
- Chawdhary, G. (2025, March 23). *What are the common mistakes in software development?* Security Compass.
<https://securitycompass.com/blog/common-mistakes-in-software-development/>
- GeeksforGeeks. (2025, October 25). *Subroutine: Nesting and stack memory*.
<https://www.geeksforgeeks.org/computer-organization-architecture/subroutine-subroutine-nesting-and-stack-memory/>
- Domas, S., & Domas, C. (2024). *x86 software reverse-engineering, cracking, and counter-measures*. Wiley.
- W3Schools. (n.d.). *C Tutorial*. <https://www.w3schools.com/c/index.php>

The background is a dark blue gradient. In the corners, there are white line-art illustrations of circuit boards or neural networks, with lines and small circles representing nodes.

THANK YOU!